React Viva Notes — Theory + Lab Progression (Lab66 → Lab75)

Topic 1: Introduction to React & SPA

Q: What is a Single Page Application (SPA)?

Theory: An application that works in a browser and does not need page reloading during its use. Examples: Gmail, Google Maps, Facebook, GitHub.

Lab context: All labs from 66-74 are SPAs created with Create React App. Navigation between views happens client-side (Lab70+ uses React Router) without page reload.

Q: What is React?

Theory: A JavaScript library by Facebook used to create single page applications. Other options: Angular, Vue JS. Key concepts: JSX and Virtual DOM.

Lab context: Each lab progressively builds React skills — from basic JSX (Lab66) to full-stack auth with mock backend (Lab75).

Q: What is JSX?

Theory: JSX stands for JavaScript XML. It is a syntax extension of JavaScript that allows us to write HTML in React. JSX gets transpiled to `React.createElement()` calls by Babel.

Lab66 proof: `<h1>Welcome to React Learning, {name}</h1>` — HTML-like syntax with `{name}` interpolation.

Q: What is Virtual DOM?

Theory: A virtual representation (a copy) of the actual DOM kept in memory. Updating the virtual DOM is faster than updating the actual DOM. React compares virtual DOM with a snapshot (diffing) and applies minimal real DOM updates.

Topic 2: Setup & Directory Structure

Q: How do you create a React app?

```
npx create-react-app hello-world
cd hello-world
npm start
```

Runs on `http://localhost:3000`.

Q: Explain the directory structure.

File/Folder	Purpose
`node_modules/`	Contains all React JS dependencies

File/Folder	Purpose
`.gitignore`	Tells git which files/folders to ignore
`package.json`	Contains dependencies and scripts
`src/`	Main folder — your code goes here
`src/index.js`	First file executed when the project runs
`src/App.js`	Root component loaded under index.js

Lab context: All labs (66-74) follow this exact structure. The progression shows how `index.js` evolves: - Lab66-69: Renders `<App />` - Lab70: `<RouterProvider router={router} />` - Lab74: `<Provider store={store}><AutoLogin><RouterProvider /></AutoLogin></Provider>`

Topic 3: Components

Q: What are components in React?

Theory: Building blocks to create the UI. Reusable pieces of code composed together to form complex UIs.

Q: Two types of components?

Type	Description
Functional Component	JavaScript function that takes props and returns JSX. Stateless (before hooks).
Class Component	ES6 class with state and lifecycle methods. Stateful.

Lab context: All labs use **functional components** exclusively. Lab68 introduces the first child component (`Child.js`). Lab70 introduces the `components/` directory pattern.

Q: Functional component example?

```
function Greeting(props) {
  return <div><h1>Hello {props.name}</h1></div>;
}
export default Greeting;
```

Q: What is the App component?

Theory: The root component that contains all other components.

Lab context: App.js is the entry component in all labs. In Lab74, App.js becomes dead code (reverted to default CRA spinning logo) because routing is handled entirely through `index.js` and `router.js`.

Topic 4: JSX Basics — Variables, CSS, Images

Q: How do you display a variable in JSX?

Use curly braces `{}`:

```
let greeting = "Hello World";
<h1>{greeting}</h1>
```

Lab66 proof: `{name}`, `{age}`, `{isStudent.toString()}` in the profile card.

Q: How do you use inline CSS in React?

```
<h1 style={{color:"red"}}>Hello World</h1>
// Or via variable:
let greetingStyle = {"color":"red"};
<h1 style={greetingStyle}>{greeting}</h1>
```

Q: Difference between external CSS (App.css) and global CSS (index.css)?

- **App.css:** Imported in `App.js` — styles apply only to components that import it
- **index.css:** Imported in `index.js` — **global** styles that work across all files

Lab66 proof: `App.css` has component-specific card styles; `index.css` has body reset styles.

Q: How do you add Bootstrap to React?

Two methods: 1. **CDN (Labs 66-73):** Add `<link>` in `public/index.html` 2. **npm (Lab74):** `npm install bootstrap` → `import 'bootstrap/dist/css/bootstrap.min.css'` in `index.js`

Q: How do you show images?

Type	Code
External	`<img src="https://..." />`
Internal	`import img from './images/photo.jpg'` → `<img src={img} />`

Lab66 proof: Both external (Pinterest URL) and internal (`eren.jpg` from `src/images/`) images displayed.

Topic 5: Data Types & Display

Q: How do you display different data types?

```
const id = 1;            // Number: {id}
const name = "John";     // String: {name}
const x = true;          // Boolean: {x.toString()}  ← needs .toString()!
const course = ["python", "java"];  // Array: {course[1]}
const student = {name:"Vijay", location:"Kochi"};  // Object: {student.name}
```

Lab67 proof: `{name}`, `{age}`, `{isStudent.toString()}` all demonstrated.

Q: Why use .toString() for booleans?

React does not render boolean values directly — they're silently ignored. `.toString()` converts `true`/`false` to the string "true"/"false".

Topic 6: Conditional Rendering

Q: How do you use if/else in JSX?

Not directly inside JSX — use ternary or declare JSX outside:

```
let showGreeting = true;
let greetingCode = showGreeting ? <h1>Hello</h1> : null;
```

OR inline: `{isBright ? "The room is bright" : "The room is dark"}`

Lab67-68 proof: Lab67 uses `if` outside JSX for list building. Lab68 uses ternary `{isBright ? "Turn OFF" : "Turn ON"}` inside JSX.

Topic 7: Lists & Keys

Q: How do you render lists?

Using `.map()` (Lab67):

```
{greetings.map((item) => <p key={index}>{item}</p>)}
```

Using `for` loop (Lab67):

```
const fruitList = [];
for (let i = 0; i < fruits.length; i++) {
  fruitList.push(<li key={i}>{fruits[i]}</li>);
}
// Then render: {fruitList}
```

Q: What is the key prop and why is it important?

- Helps React identify which items changed, added, or removed
- Should be a unique and stable identifier (prefer ID over index)
- Without keys, React re-renders the entire list

Lab71 proof: `key={item.id}` used in CRUD table — proper unique key. **Lab67 proof:** `key={i}` used — acceptable for static lists only.

Topic 8: Events

Q: How do you handle events in React?

```
<button onClick={() => console.log("welcome")}>Click</button>
// Or function reference:
function handleClick() { console.log('Clicked!'); }
<button onClick={handleClick}>Click</button>
```

Lab67 proof: `onClick={showMessage}` on the "Show Enthusiasm" button. **Key rule:** Pass function reference, not function call — `onClick={handleClick}` NOT `onClick={handleClick()}`.

Q: Other events?

Change `onClick` to: `onDoubleClick`, `onMouseEnter`, `onChange`, `onSubmit`, etc.

Topic 9: State (useState)

Q: What is state in React?

Theory: An object that stores dynamic data that changes over time and affects component behavior. Functional components use the `useState()` hook.

Q: How does useState work?

```
import { useState } from 'react';
const [count, setCount] = useState(0);
// count = current state (0), setCount = updater function
setCount(count + 1);  // triggers re-render
```

Lab68 proof: `const [isBright, setIsBright] = useState(false)` — toggles room light. **Lab71 proof:** 5 state variables manage CRUD operations.

Q: What happens when state is updated?

React re-renders the component with the new state value. The UI updates automatically.

Topic 10: Forms & Controlled Inputs

Q: What is a controlled component?

An input whose value is controlled by React state:

```
const [name, setName] = useState('');
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Lab71 proof: `value={itemName} onChange={handleInputChange}` in add book form.

Q: How do you handle form submission?

```
const handleSubmit = (e) => {
  e.preventDefault();  // prevents page reload
  // process data
};
<form onSubmit={handleSubmit}>...</form>
```

Topic 11: Form Validation (Lab71 homework-app)

Q: How do you implement form validation in React?

Store validation errors as an **error object** keyed by field name:

```
const [errors, setErrors] = useState({});

function validateForm() {
  const newErrors = {};
  if (!name.trim()) newErrors.name = 'Name is required';
  if (!email.trim()) newErrors.email = 'Email is required';
  if (duplicateRollNumber(roll)) newErrors.roll = 'Roll number already exists';
  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
}
```

Lab71 homework-app proof: `validateForm()` returns an error object with per-field messages, checked before submission.

Q: How do you display validation errors per field?

```
{errors.name && <span className="error">{errors.name}</span>}
{errors.roll && <span className="error">{errors.roll}</span>}
```

Q: How do you clear errors as the user types?

```
function handleChange(e) {
  setFormData({ ...formData, [e.target.name]: e.target.value });
  if (errors[e.target.name]) {
    setErrors({ ...errors, [e.target.name]: undefined });
  }
}
```

This provides **live error clearing** — the error disappears as soon as the user starts typing in that field.

Q: How do you validate passwords match?

```
if (password !== confirmPassword) {
  setError('Passwords do not match');
```

```
  return;
}
```

Lab73 homework-app proof: Register form includes password confirmation field.

Q: What are common validation patterns?

Pattern	Description
Required fields	Check for empty/whitespace strings
Duplicate check	Search existing data for conflicts
Password match	Compare password vs confirmPassword
Live error clearing	Remove field error on keystroke
Error object	`{ fieldName: message }` — enables per-field error display

Topic 12: Props & Component Communication

Q: What are props?

Theory: Short for "properties" — a way to pass data from one component to another. Props are **read-only** — child cannot modify them.

Q: How does parent pass data to child?

```
// Parent (App.js)
<Child name={name} age={age} />

// Child (Child.js)
function Child(props) {
  return <p>Name: {props.name}, Age: {props.age}</p>;
}
```

Lab68 proof: `<Child isBright={isBright} toggleLight={toggleLight} />`

Q: How does child communicate back to parent?

Parent passes a **function reference** as prop, child calls it:

```
// Parent
function incrementCount() { setCount(count + 1); }
<Child increment={incrementCount} />

// Child
<button onClick={props.increment}>Increment</button>
```

Lab68 proof: Child component calls `toggleLight` prop to update parent's state.

Q: What is props.children?

Used to render child elements nested inside a component:

```
<Child name="Alice"> <p>Welcome</p> </Child>
// Child accesses: {props.children}
```

Lab74 proof: `<ProtectedRoute>` and `<AutoLogin>` use `{children}` pattern to wrap components.

Topic 12: useEffect

Q: What is useEffect?

Theory: A hook to manage side effects in functional components. Allows performing actions **after** the component has rendered or re-rendered.

Q: Three dependency array patterns?

Dependency	Behavior
`[]` (empty)	Runs only once on mount
No array	Runs after every render
`[count]`	Runs only when `count` changes

Q: Examples?

On mount only (Lab69, Lab72):

```
useEffect(() => { loadBooks(); }, []);
```

On state change (Lab69):

```
useEffect(() => {
  console.log('User changed to ' + user);
}, [user]);
```

On every render:

```
useEffect(() => { console.log("rendered"); });
// No dependency array
```

Q: What happens in StrictMode?

In development, effects run twice to detect bugs. This is intentional.

Topic 13: Routing (React Router DOM)

Q: What is React Router DOM?

Theory: A third-party library for handling routing in SPAs. It dynamically changes page content based on URL without page reload.

Q: How do you set up routes?

Using createBrowserRouter (Lab70, Lab74):

```
const router = createBrowserRouter([
  { path: '', element: <App/> },
  { path: 'aboutus', element: <Aboutus/> },
  { path: 'student/:name', element: <StudentProfile/> },
]);
// index.js: <RouterProvider router={router} />
```

Using BrowserRouter + Routes (Lab73):

```
<BrowserRouter>
  <Routes>
    <Route path="/register" element={<Register />} />
    <Route path="/login" element={<Login />} />
  </Routes>
</BrowserRouter>
```

Q: How do you navigate between pages?

Method	Component/Hook	Usage
Declarative	`<Link to="/path">`	Renders `<a>` with SPA navigation
Declarative (active)	`<NavLink to="/path">`	Provides `isActive` class for styling
Imperative	`useNavigate()`	`navigate('/path')` in event handlers

Q: What are parameterized routes?

```
// Route: { path: 'student/:name', element: <StudentProfile/> }
// Component:
const { name } = useParams();
<h1>Welcome, {name}!</h1>
// URL: /student/Eren%20Yeager → renders "Welcome, Eren Yeager!"
```

Lab70 proof: Student profile page uses `useParams` to read student name from URL.

Topic 14: CRUD Operations (Local State)

Q: What does CRUD stand for?

Create, Read, Update, Delete

Q: How do you implement CRUD in React?

Create (Lab71):

```
const newItem = { id: items.length + 1, name: itemName };
setItems([...items, newItem]);
setItemName("");  // reset form
```

Read:

```
{items.map((item) => <tr key={item.id}><td>{item.name}</td></tr>)}
```

Update:

```
const updatedItems = items.map((item) =>
  item.id === editingId ? { ...item, name: newName } : item
);
setItems(updatedItems);
```

Delete:

```
setItems(items.filter((item) => item.id !== idToDelete));
```

Q: What is derived state?

Data computed from existing state:

```
const filteredItems = items.filter(item =>
  item.name.toLowerCase().includes(searchTerm.toLowerCase())
);
```

No separate state needed — recalculated on every render.

Topic 15: API Integration with fetch

Q: How do you fetch data from an API?

```
useEffect(() => { fetchPosts(); }, []);

async function fetchPosts() {
  const res = await fetch('https://api.example.com/posts');
  const data = await res.json();
  setPosts(data);
}
```

Lab72 proof: `api.js` has `fetchBooks()`, `createBook()`, `updateBook()`, `deleteBook()` using native `fetch()`.

Q: How do you handle loading and error states?

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

async function loadBooks() {
  try {
    setLoading(true);
```

```
    setError(null);
    const data = await fetchBooks();
    setBooks(data);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
}
```

Q: What HTTP methods correspond to CRUD?

Operation	HTTP Method	fetch Example
Create	POST	`fetch(url, { method: 'POST', body: JSON.stringify(data) })`
Read	GET	`fetch(url)`
Update	PUT	`fetch(url/id, { method: 'PUT', body: JSON.stringify(data) })`
Delete	DELETE	`fetch(url/id, { method: 'DELETE' })`

Topic 16: Axios

Q: What is Axios?

Theory: A popular JavaScript library for making HTTP requests. Promise-based. Supports async/await. Automatically parses JSON.

Q: How do you make requests with Axios?

```
import axios from 'axios';

axios.get('url', { headers: {...} })
  .then(response => console.log(response.data))
  .catch(error => console.log(error));

axios.post('url', { key: 'value' }, { headers: {...} })
  .then(response => ...)
  .catch(error => ...);
```

Q: Axios request patterns?

GET with query params:

```
axios.get('url', { params: { key: 'value' } })
```

POST with JSON body (Lab73-74):

```
axios.post('https://api.example.com/login', { email, password })
```

POST with FormData:

```
const data = new FormData();
data.append('email', 'demo@example.com');
axios.post('url', data)
```

Q: Difference between fetch and axios?

Aspect	fetch	axios
JSON parsing	Manual: `res.json()`	Automatic
Error handling	Only rejects on network error	Rejects on any non-2xx
Request headers	Manual	Built-in
Lab used	Lab72	Lab73, Lab74

Topic 17: Authentication

Q: How does authentication work in React?

1. User submits credentials (email/password)
2. Login API verifies and returns a **token**
3. Token is stored (in Redux, localStorage, or both)
4. Token is sent with every subsequent API request as `Authorization: Bearer {token}`
5. On logout, token is cleared

Q: Registration flow (Lab73-74)?

```
axios.post('/api/register', { user_name, email, password })
  .then(response => navigate('/login'))
  .catch(error => {
    if (error.response.data.errors) {
      setErrorMessage(Object.values(error.response.data.errors).join(' '));
    }
  });
```

Q: Login flow (Lab73-74)?

```
axios.post('/api/login', { email, password })
  .then(response => {
    const user = { email, token: response.data.token };
    dispatch(setUser(user));                    // save to Redux
    localStorage.setItem('user', JSON.stringify(user)); // persist
    navigate('/dashboard');
  })
  .catch(error => {
```

```
    // Handle validation errors, server errors, network errors
  });
```

Q: How do you handle API errors properly?

```
.catch(error => {
  if (error.response && error.response.data.errors) {
    // Validation errors (object) → flatten to string
    setErrorMessage(Object.values(error.response.data.errors).join(' '));
  } else if (error.response && error.response.data.message) {
    // Server message (string)
    setErrorMessage(error.response.data.message);
  } else {
    // Network error (no response)
    setErrorMessage('Failed to connect to API');
  }
});
```

Topic 18: Redux Toolkit

Q: What is Redux?

Theory: A predictable state management library. Maintains entire application state in a single immutable **store**. State is read-only — changes are made by dispatching **actions** through **reducers** (pure functions).

Q: Key Redux concepts?

Concept	Description
Store	Single source of truth holding entire app state
Action	Plain JS object describing what happened (`{ type: 'setUser', payload: {...} }`)
Reducer	Pure function taking current state + action → returns new state
Dispatch	Sending an action to update the store

Q: What is Redux Toolkit createSlice?

```
import { createSlice } from "@reduxjs/toolkit";

const authSlice = createSlice({
  name: 'auth',
  initialState: { user: null },
  reducers: {
    setUser: (state, action) => { state.user = action.payload },
    removeUser: (state) => { state.user = null },
```

```
  }
});

export const { setUser, removeUser } = authSlice.actions;
export default authSlice.reducer;
```

Lab74 proof: `src/store/authSlice.js` — full implementation with `setUser` and `removeUser`.

Q: How does Immer work in RTK?

Allows writing mutable-looking code — `state.user = action.payload` — Immer converts it to immutable updates behind the scenes.

Q: How do you configure the store?

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from './authSlice';

const store = configureStore({
  reducer: { auth: authReducer }
});
```

Q: How do you configure multiple Redux slices (Lab74 homework-app)?

Real-world apps have multiple slices for different features:

```
// store.js
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '../features/auth/authSlice';
import productReducer from '../features/product/productSlice';

export const store = configureStore({
  reducer: { auth: authReducer, product: productReducer },
});
```

```
// productSlice.js - second slice
const productSlice = createSlice({
  name: 'product',
  initialState: { items: [], loaded: false },
  reducers: {
    setProducts: (state, action) => {
      state.items = action.payload;
      state.loaded = true;
    },
  },
});
```

Components select from specific slices:

```
const user = useSelector(state => state.auth.user);
const products = useSelector(state => state.product.items);
```

Q: How do you look up an entity from Redux store?

```
const product = useSelector((state) =>
  state.product.items.find((p) => String(p.id) === String(id))
);
```

Lab74 homework-app proof: ViewProduct page finds a product by id from the Redux store rather than making a new API call.

Q: How do you use Redux in components?

Provider (index.js):

```
<Provider store={store}>
  <App />
</Provider>
```

useSelector — read state:

```
const user = useSelector(store => store.auth.user);
```

useDispatch — update state:

```
const dispatch = useDispatch();
dispatch(setUser({ email, token }));
```

Lab74 proof: Navbar uses `useSelector` to check auth state and conditionally show Login/Logout. Login component uses `useDispatch` to save user.

Topic 19: Persistence & Auto-Login

Q: How do you persist auth across page refresh?

Save to **localStorage** when logging in:

```
localStorage.setItem('user', JSON.stringify({ email, token }));
```

Q: How do you restore auth on app start?

AutoLogin component (Lab74):

```
function AutoLogin({ children }) {
  const dispatch = useDispatch();
  useEffect(() => {
    const user = localStorage.getItem('user');
    if (user) dispatch(setUser(JSON.parse(user)));
  }, [dispatch]);
  return children;
}
```

Wrapped in index.js: `<Provider><AutoLogin><RouterProvider /></AutoLogin></Provider>`

Topic 20: Protected Routes

Q: How do you restrict access to logged-in users?

Create a wrapper component:

```
function ProtectedRoute({ children }) {
  const user = useSelector(store => store.auth.user);
  if (!user) return <Navigate to="/login" replace />;
  return children;
}
```

Usage in router: `{ path: '/products', element: <ProtectedRoute><ProductList /></ProtectedRoute>`
`}`

Lab74 proof: `ProtectedRoute.js` guards the `/products` page.

Q: Alternate pattern using Higher-Order Component (HOC)?

```
export const checkAuth = (Component) => {
  function Wrapper(props) {
    const user = useSelector(store => store.auth.user);
    const navigate = useNavigate();
    useEffect(() => { if (!user) navigate('/login'); }, [user]);
    return <Component {...props} />;
  }
  return Wrapper;
};
// Usage: export default checkAuth(ListPost);
```

Topic 21: Higher-Order Component (HOC) Pattern (Lab75 homework-app)

Q: What is a Higher-Order Component (HOC)?

Theory: A function that takes a component and returns a new enhanced component. It's a pattern for reusing component logic — an alternative to wrapper components like `<ProtectedRoute>`.

Q: How do you implement auth protection using HOC?

```
// checkAuth.js
import { useSelector } from 'react-redux';
import { Navigate } from 'react-router-dom';

function checkAuth(Component) {
  return function AuthenticatedComponent(props) {
    const { token } = useSelector((state) => state.auth);
    if (!token) {
      return <Navigate to="/login" replace />;
    }
    return <Component {...props} />;
```

```
  };
}
export default checkAuth;
```

Usage:

```
const ProtectedStudentList = checkAuth(StudentList);

// In router:
<Route path="/" element={<ProtectedStudentList />} />
```

Q: HOC vs ProtectedRoute wrapper — what's the difference?

Pattern	Approach	Lab
Wrapper component	`<ProtectedRoute><Component /></ProtectedRoute>`	Lab74 classwork, Lab75 classwork
HOC	`const Protected = checkAuth(Component)` then `<Protected />`	Lab75 homework

HOCs are an older but still widely used React pattern. The wrapper component approach is more modern but both achieve the same goal.

Q: How does Redux localStorage re-hydration work with HOCs?

The homework-app uses a different pattern than the classwork:

Lab75 classwork: `initialState` reads from localStorage directly in the slice. **Lab75 homework:** An `AutoLogin` component dispatches `setUserFromLocalStorage` on mount:

```
// authSlice.js
setUserFromLocalStorage(state) {
  const savedUser = localStorage.getItem('user');
  const savedToken = localStorage.getItem('token');
  if (savedUser && savedToken) {
    state.user = JSON.parse(savedUser);
    state.token = savedToken;
  }
},

// AutoLogin.js
function AutoLogin({ children }) {
  const dispatch = useDispatch();
  useEffect(() => { dispatch(setUserFromLocalStorage()); }, [dispatch]);
  return children;
}
```

Both patterns work — the slice-direct approach is simpler, the AutoLogin approach is more explicit.

Topic 22: Token in API Requests

Q: How do you send the auth token?

Set the `Authorization` header as a Bearer token:

```
axios.get('https://api.example.com/products', {
  headers: { 'Authorization': 'Bearer ' + user.token }
});
```

Lab74 proof: `ProductList.js` sends Bearer token when fetching products.

Topic 23: json-server & Mock Backend (Lab75)

Q: What is json-server?

Theory: A Node.js library that creates a full REST API from a JSON file. Useful for prototyping and testing frontend apps without building a real backend.

Lab75 proof:

```
// db.json - acts as the database
{
  "users": [
    { "id": 1, "name": "John Doe", "email": "john@example.com", "password": "Password123" }
  ],
  "students": [
    { "id": 1, "name": "Alice Johnson", "age": 20 },
    { "id": 2, "name": "Bob Smith", "age": 22 }
  ]
}
```

Q: How do you create a custom json-server with auth endpoints?

```
// server.js
const jsonServer = require('json-server');
const server = jsonServer.create();
const router = jsonServer.router('db.json');

server.use(jsonServer.bodyParser);

// Custom register endpoint
server.post('/api/register', (req, res) => {
  const { name, email, password } = req.body;
  const existing = router.db.get('users').find({ email }).value();
  if (existing) {
    return res.status(400).json({ message: 'Email already registered' });
  }
  const user = router.db.get('users').insert({ name, email, password }).write();
  res.json({ message: 'Registration successful', user });
```

```
});

// Custom login endpoint
server.post('/login', (req, res) => {
  const { email, password } = req.body;
  const user = router.db.get('users').find({ email, password }).value();
  if (!user) {
    return res.status(401).json({ message: 'Invalid email or password' });
  }
  const token = 'mock-token-' + user.id + '-' + Date.now();
  res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
});

server.use(router);
server.listen(3001);
```

Lab75 proof: `server.js` and `db.json` in the project root.

Q: How does Redux initialization differ in Lab75 vs Lab74?

Lab74: AutoLogin component reads localStorage on mount and dispatches `setUser`. **Lab75:** `initialState` reads from localStorage directly in the slice:

```
const initialState = {
  user: localStorage.getItem('user') ? JSON.parse(localStorage.getItem('user')) : null,
  token: localStorage.getItem('token') || null,
};

const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    login(state, action) {
      state.user = action.payload.user;
      state.token = action.payload.token;
      localStorage.setItem('user', JSON.stringify(action.payload.user));
      localStorage.setItem('token', action.payload.token);
    },
    logout(state) {
      state.user = null;
      state.token = null;
      localStorage.removeItem('user');
      localStorage.removeItem('token');
    },
  },
});
```

No separate `AutoLogin` wrapper component needed — state is hydrated synchronously on store creation.

Q: What is the pages directory pattern?

Lab75 proof: Components are split into: - `src/components/` — Reusable UI (Navbar, ProtectedRoute) - `src/pages/` — Route-level pages (Login, Register, StudentList) - `src/store/` — Redux state management

This is cleaner than Lab74 where everything was in `src/components/`.

Q: How does the full-stack flow work in Lab75?

```
User registers → POST /api/register → user saved in db.json → redirect to /login
User logs in  → POST /login → mock token returned → stored in Redux + localStorage
Student list  → GET /students (with Bearer token) → protected data displayed
Logout        → clear Redux + localStorage → redirect to /login
```

Topic 24: Complete Feature Comparison

Lab	Topic	Classwork Concepts	Homework Concepts (Additions)
66	JSX Basics	JSX, variables, images, Bootstrap CDN	Same concepts
67	Lists & Events	.map(), for loop, onClick, key prop	**Passing params to event handlers** `onClick={() => fn(item)}`
68	State & Props	useState, child components, props	**Controlled inputs, array state with spread, multiple useState**
69	Side Effects	useEffect, dependency array	Same concepts
70	Routing	createBrowserRouter, Link, useParams, useNavigate	**NavLink `end` prop** for active styling
71	Local CRUD	Controlled forms, immutable array ops, edit/cancel	**Form validation error object, duplicate check, inline table editing, live error clearing**

Lab	Topic	Classwork Concepts	Homework Concepts (Additions)
72	API CRUD	fetch(), async/await, loading/error states	Same concepts
73	Auth (axios)	Register/login, .then/.catch, error handling	**Password confirmation, custom per-component CSS, `<Navigate>` redirect**
74	Redux Toolkit	createSlice, Provider, useSelector/useDispatch, localStorage, AutoLogin	**Multiple slices (auth+product), .jsx convention, store entity lookup, inline-only styling**
75	Full-Stack Auth	json-server, server.js, db.json, mock API, pages/	**HOC pattern `checkAuth(Component)`, Redux localStorage re-hydration action, AutoLogin mount-dispatch**

Quick Reference: Key Code Patterns

useState

```
const [state, setState] = useState(initialValue);
```

useEffect

```
useEffect(() => { /* side effect */ }, [dependencies]);
```

Controlled Input

```
<input value={state} onChange={(e) => setState(e.target.value)} />
```

Immutable Array Update

```
setItems([...items, newItem]);                          // Create
setItems(items.map(i => i.id === id ? new : i));        // Update
setItems(items.filter(i => i.id !== id));               // Delete
```

API Call with fetch

```
async function fetchData() {
  const res = await fetch(url);
  if (!res.ok) throw new Error('Failed');
  return res.json();
}
```

API Call with axios

```
axios.post(url, data)
  .then(res => console.log(res.data))
  .catch(err => console.log(err.response.data));
```

Redux Slice

```
createSlice({ name: 'x', initialState: { key: val }, reducers: { action: (s, a) => { s.key = a
```

Router

```
createBrowserRouter([{ path: '/', element: <Comp /> }]);
```

Protected Route

```
function Protected({ children }) {
  const user = useSelector(s => s.auth.user);
  return user ? children : <Navigate to="/login" />;
}
```

json-server (Lab75)

```
// server.js - mock backend
const jsonServer = require('json-server');
const server = jsonServer.create();
const router = jsonServer.router('db.json');
server.use(jsonServer.bodyParser);
server.post('/login', (req, res) => { /* validate, return token */ });
server.use(router);
server.listen(3001);
```

Redux Slice with localStorage

```
const initialState = {
  user: JSON.parse(localStorage.getItem('user')),
  token: localStorage.getItem('token'),
```

```
};
const slice = createSlice({
  name: 'auth', initialState,
  reducers: {
    login: (s, a) => { s.user = a.payload.user; s.token = a.payload.token; localStorage.setIte
    logout: (s) => { s.user = null; s.token = null; localStorage.removeItem('token'); },
  },
});
```

Form Validation with Error Object (Lab71 homework)

```
const [errors, setErrors] = useState({});
function validate() {
  const e = {};
  if (!name) e.name = 'Required';
  if (duplicateRoll) e.roll = 'Duplicate';
  setErrors(e);
  return Object.keys(e).length === 0;
}
// Clear on keystroke:
function handleChange(e) {
  setForm({ ...form, [e.target.name]: e.target.value });
  if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: undefined });
}
```

Multiple Redux Slices (Lab74 homework)

```
const store = configureStore({
  reducer: { auth: authReducer, product: productReducer },
});
// Component:
const products = useSelector(state => state.product.items);
const product = useSelector(state => state.product.items.find(p => p.id === id));
```

HOC Pattern (Lab75 homework)

```
function checkAuth(Component) {
  return function Authenticated(props) {
    const { token } = useSelector(state => state.auth);
    if (!token) return <Navigate to="/login" replace />;
    return <Component {...props} />;
  };
}
// Usage: const ProtectedPage = checkAuth(MyPage);
```

NavLink with end prop (Lab70 homework)

```
<NavLink to="/" end>Home</NavLink>
// end prevents matching / as active for all sub-routes
```

Password Confirmation (Lab73 homework)

```
if (password !== confirmPassword) {
  setError('Passwords do not match');
  return;
}
```